1 What Would Python Display? (14pts)

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is valued. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **`Error`**, but include all output displayed before the error.
- To display a function value, write **`Function`**. To display an iterator value, write **`Iterator`**.
- If an expression would take forever to evaluate, write **`Forever`**.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.
Assume that you have first started `python3` and executed the statements on the left.

```
def stay_up_late(todos, hours_per_day=24):
    sleep_time = 8
    def finish_one_by_one(todos, remain_time):
        if remain_time < sleep_time:
            return True
        if not todos and remain_time >= sleep_time:
            return False
        else:
            todo, todos = todos[0], todos[1:]
            return stay_up_late(todos, remain_time - sum(todo))
    return finish_one_by_one(todos, hours_per_day)

def time_sequence(todos, rest):
    for todo in todos:
        yield sum(todo) + rest

study  = [1, 2, 3, 4]
relax  = [1] * 4
games  = [1, 4]
meals  = [1] + [1] + [1]
todos  = [study, relax, games, meals]
```

Expression	Interactive Output
`pow(2, 11) - 26`	`2022`
`print(4, 5) + 1`	`4 5` `Error`

`f1 = print(2+3)`
`f2 = lambda: print(2+3)`
`print(f1, f2, f2())`	

`print(sum(study))`
`print(len(meals))`
`print(len(todos))`	

`dropped = True and ((True or False) or True)`
`good_night = dropped or not stay_up_late`
`print(dropped, good_night)`	

`print(stay_up_late(todos))`
`print(stay_up_late(todos, 48))`
`print(stay_up_late([meals]))`	

`tiktok = time_sequence(todos, 0)`
 	
`for todo in tiktok:`
`  if todo >= 3:`
`  print(next(tiktok))`
`  else:`
`  print(todo)`
 	
`alltime = 0`
`for todo in tiktok:`
`  alltime += todo`
 	
`print(alltime)`	

2 Lists (22pts)

Which expressions in the table can get the cake? Suppose we started `python3` and executed the statements below. For each expression in the table, if evaluating it returns the string "cake", write **T**. Otherwise, write **F**. (6pts)

```
lst = ["trap", [lambda: "cake"]]
lst[1].append(lst)
```

Expression	Your Answer
`(lambda: "cake")()`	**T**
`(lambda x: "cake")()`	**F**
`lst[0]`	
`lst[1][0]`	
`lst[1][0]()`	
`lst[1:][0]()`	
`(lst+[lst])[2][1][0]()`	
`lst.pop()[1][1][0]()`	

Write expressions involving the list `l` that evaluate to the following values.
Each expression shall be **a single list comprehension** involving the list `l`. (4pts)

```
>>> l = [[1,2], [3,4], [5,6], [7,8], [9,10]]
>>> [x for x, y in l if y < 8]  # an example for you
>>> [1, 3, 5]
>>> _____________________________________
[[10,9], [8,7], [6,5], [4,3], [2,1]]
>>> _____________________________________
[2,4]
```

Draw the environment diagrams for the following two pieces of code.

Diagram #1 (6pts)

```
a = [1, 2, [3, 4]]
b = a
c = a[:]
a[1] = b[0]
c[2][0] = 5
```

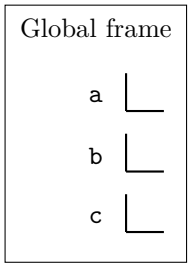
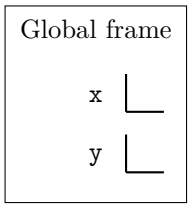


Diagram #2 (6pts)

```
x = [False, True, False]
y = []
for _ in range(5):
    if x[0]:
        x = [x.append([x[0]])] + x[1:]
    else:
        x = x[1:]
    y.append(x[0])
```

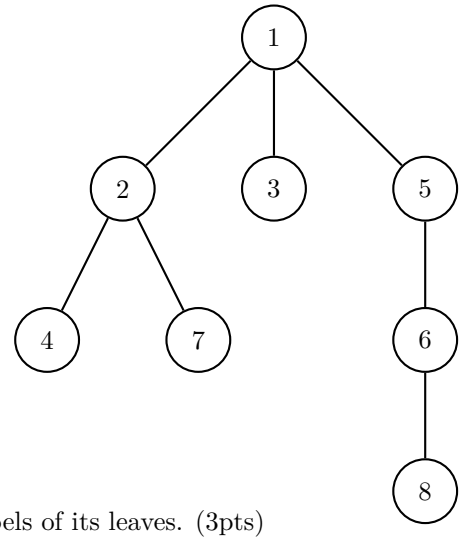


3 Trees (23pts)

This problem is based on an assumption that all labels in a tree are different.

The following tree APIs are available for this problem:

- `tree(label, branches=[])`
- `label(tree)`
- `branches(tree)`
- `is_leaf(tree)`



Define the tree in the figure using `tree` constructor. (3pts)

```
t1 =
```

Implement `sum_leaves`, which takes in a tree `t` and return sum of labels of its leaves. (3pts)

```
def sum_leaves(t):
    """ Returns sum of labels of t's leaves.
    >>> sum_leaves(t1)  # 4 + 7 + 3 + 8
    22
    """
    if _____________:
        return _____________
    return sum([____________________ for _____ in _____________])
```

The depth of a node `n` in a tree `t` is defined recursively as follows:

- If `n` is the root of `t`, its depth is 0.
- Otherwise, depth of `n` is its parent's depth plus 1.

Implement `depth`, which takes in a tree and a label. It returns the depth of the node with that label or `None`. (6pts)

```
def depth(t, v):
    """ Returns the depth of the node with given label, or None if not found.
    >>> depth(t1, 1), depth(t1, 3), depth(t1, 8), depth(t1, 9)
    (0, 1, 3, None)
    """
    if ____________________:
        return ________
    for _____ in _____________:
        d = _____________
        if d is not None:
            return ________
    return ________
```

Implement `sum_lowest_leaves`, which takes in a tree `t` and returns the sum of labels of the tree's lowest leaves (lowest leaves are leaf nodes with the highest depth). (4pts)

```
def sum_lowest_leaves(t):
    """ Returns the sum of labels of t's lowest leaves.
    >>> sum_lowest_leaves(t1)
    8
    >>> sum_lowest_leaves(branches(t1)[0])  # 4 + 7
    11
    >>> sum_lowest_leaves(tree(1, [tree(2), tree(3), tree(4)]))
    9
    """
    def helper(trees):
        if all(_____________________________________):
            return _____________________________________
        return helper(sum(_____________________________________, _____))
    return helper(_____)
```

Finally, implement `max_path`, which takes in a tree `t` and a positive integer `k`, and returns the highest label sum of paths in `t` with at most `k` nodes. The path may not start at the root of `t` but each step should be going downwards (from top to bottom, from root to leaf). You may assume that labels can be negative. (7pts)

```
def max_path(root, k):
    """ Returns the highest label sum of paths with length no more than k.
    >>> max_path(t1, 2)  # 6 + 8
    14
    >>> max_path(t1, 4)  # 1 + 5 + 6 + 8
    20
    >>> max_path(tree(1, [tree(10, [tree(20, [tree(5)])])]), 2)
    30
    >>> max_path(tree(5, [tree(-1, [tree(-2, [tree(-3)])])]), 2)
    5
    """
    def helper(t, k, flag):
        if ____________:
            return 0
        elif ____________________:
            return label(t)
        max1 = max(_____________________________________________________________)
        if ____________:
            ____________________
        max2 = max(_____________________________________________________________)
        return ______________________________
    return helper(root, k, False)
```